

Question-1:

Story:

A long time ago, Josephus and about 40 soldiers were trapped by the Romans.

They thought they would be captured, so they decided to kill themselves instead.

They stood in a circle and decided that every third person would be killed.

Josephus did not want to die. He figured out where he should stand so that he would survive.

In the end, only Josephus. Josephus surrendered himself to the Romans instead of committing suicide.

This story later inspired the Josephus Problem in computer science.

Problem Statement:

There are **n people** standing in a circle, numbered from **1 to n**.

Starting from person **1**, count people clockwise. Every **k-th person** is removed from the circle. After someone is removed, counting continues from the **next remaining person**.

This process continues until only **one person** is left.

Given **n** and **k**, determine the **1-based position** of the person who survives.

Example 1

Input: $n = 5, k = 2$

Output: 3

Explanation:

The people are eliminated in this order:

$2 \rightarrow 4 \rightarrow 1 \rightarrow 5$

Therefore, person **3** is the last survivor.

Example 2

Input: $n = 7, k = 3$

Output: 4

Explanation:

The elimination order is:

$3 \rightarrow 6 \rightarrow 2 \rightarrow 7 \rightarrow 5 \rightarrow 1$

Therefore, person **4** survives.

Constraints

$1 \leq n, k \leq 500$

Solution:

```
#include <iostream>

using namespace std;

int josephus(int n, int k) {

    // Base case

    if (n == 1)

        return 0;

    // Recursive case

    return (josephus(n - 1, k) + k) % n;

}

int main() {

    int n = 5;
```

```
int k = 2;

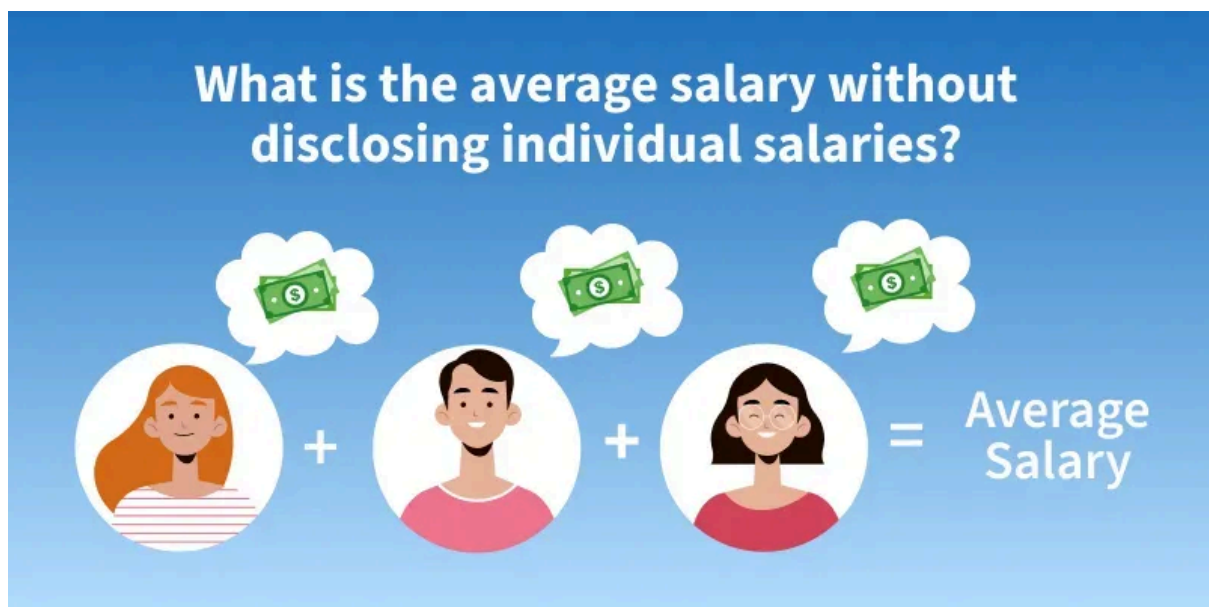
int survivor = josephus(n, k);

// Convert 0-based index to 1-based position
cout << "Survivor = " << survivor + 1 << endl;

return 0;
}
```

Question-2:

Three Employees want to know the average of their salaries. They are not allowed to share their individual salaries.



Solution:

Step 1: X adds a Random Number and his salary and tells the sum to Y.

Step 2: Y also adds a Random Number and his salary to the sum told by X and tells new sum to Z.

Step 3: Z also adds a Random Number and his salary to the sum told by Y and tells new sum to X.

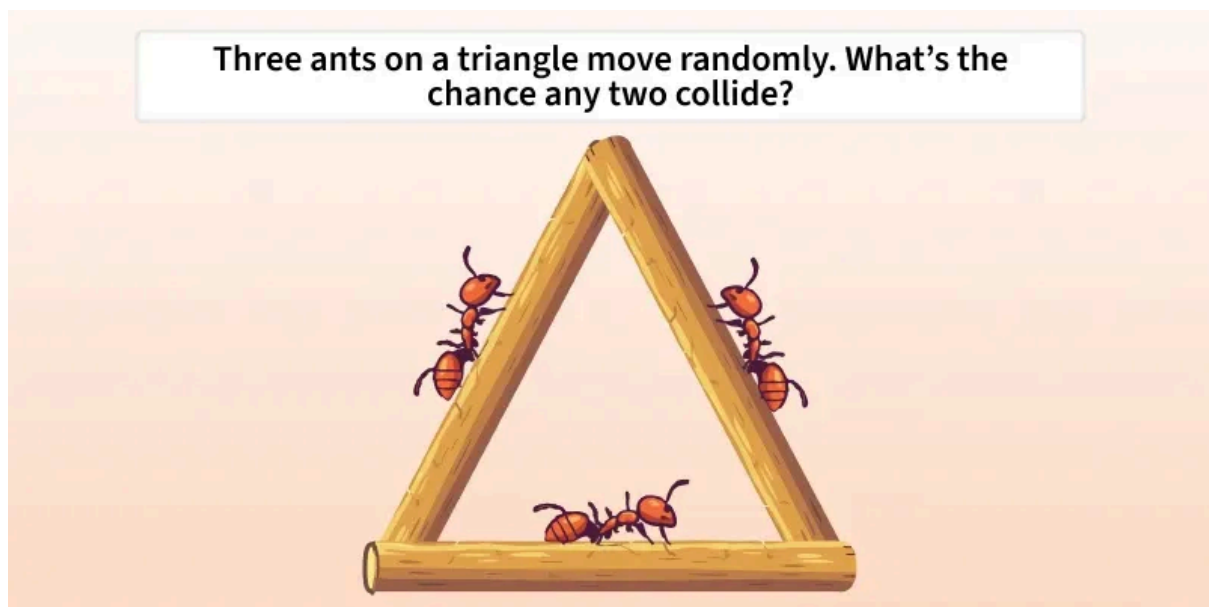
Step 4: X subtracts its random number from the sum told by Z and tells the new number to Y.

Step 5: Y subtracts its random number from the sum told by X and tells the new number to Z.

Step 6: Z subtracts its random number from the sum told by Y and announces the new number. The new number is now the sum of three salaries and the average can be calculated by dividing the sum by 3. Finally, nobody knows the salary of others, but all know average.

Question-3:

There are three ants sitting on the three corners of a triangle. All ants randomly pick a direction and start moving along the edge of the triangle. What is the probability that any two ants collide?



Solution:

Since every ant has two choices (pick either of two edges going through the corner on which ant is initially sitting), there are total 2^3 possibilities.

Out of 2^3 possibilities, only 2 don't cause collision.

So, the probability of collision is $6/8$ and the probability of non-collision is $2/8$.

Question-4:

You are given an integer N representing the size of a square chessboard of dimensions $N \times N$.

A Knight is placed at the top-left corner $(0, 0)$ of the board. Your task is to find a valid Knight's Tour, in which the Knight visits every cell exactly once.

A Knight can move in an L-shaped pattern: two cells in one direction followed by one cell perpendicular to that direction.

For example, if the Knight is currently at $(2, 2)$, its possible moves are:

$(4,3), (4,1),$

$(0,3), (0,1),$

$(3,4), (3,0),$

$(1,4), (1,0)$

The Knight must remain within the boundaries of the chessboard and cannot visit the same cell more than once.

Task

Return a matrix representing the order in which the cells are visited.

The starting cell $(0, 0)$ must contain 0 , the next visited cell must contain 1 , the following cell 2 , and so on until all $N \times N$ cells have been visited.

If a valid Knight's Tour exists, return any one valid tour.

If no valid tour is possible, return an empty matrix.

Examples

Example 1

Input:

$N = 5$

Output:

```
[ 0, 11, 2, 17, 20],  
[ 3, 16, 19, 12, 7],  
[10, 1, 6, 21, 18],  
[15, 4, 23, 8, 13],  
[24, 9, 14, 5, 22]  
]
```

Explanation:

The Knight starts at $(0, 0)$, represented by 0 .

It then moves to the cell containing 1 , followed by $2, 3$, and so on.

The final cell visited contains 24, meaning all 25 cells of the 5 × 5 board have been visited exactly once.

Since every consecutive number represents a valid Knight move, this is a valid Knight's Tour.

Example 2

Input:

N = 4

Output:

[]

Explanation:

A valid Knight's Tour cannot be completed on a 4 × 4 chessboard when starting from (0, 0).

Therefore, the function should return an empty matrix.

Example 3

Input:

N = 1

Output:

```
[  
  [0]  
]
```

Explanation:

The board contains only one cell. Since the Knight starts at $(0, 0)$, it has already visited the entire board.

Therefore, the tour is simply:

0

Constraints

$$1 \leq N \leq 6$$

Important Conditions

- The Knight must start from $(0, 0)$.
- Every cell must be visited exactly once.
- A Knight can only make valid Knight moves.
- The Knight cannot move outside the board.
- The returned matrix must contain every number from 0 to $N \times N - 1$ exactly once.
- If no valid tour exists, return an empty matrix.
- If multiple solutions exist, any valid solution is acceptable.

Question-5:

You are given n disks stacked on a starting rod (**from**). The disks are arranged such that the smallest disk is on top and the largest disk is at the bottom.

There are three rods:

- **from** — the starting rod
- **to** — the destination rod
- **aux** — the auxiliary rod

Your task is to determine the minimum number of moves required to transfer all n disks from the **from** rod to the **to** rod.

Rules

1. Only one disk can be moved at a time.
2. A disk can only be placed on an empty rod or on top of a larger disk.
3. All disks must eventually be moved from the starting rod to the target rod.

Return the minimum number of moves required.

Examples

Input:

$n = 2$

Output:

3

Explanation:

The disks can be transferred in 3 moves:

Move disk 1 from rod 1 → rod 2

Move disk 2 from rod 1 → rod 3

Move disk 1 from rod 2 → rod 3

Input:

$n = 3$

Output:

7

Explanation:

The disks can be transferred in 7 moves:

Move disk 1 from rod 1 → rod 3

Move disk 2 from rod 1 → rod 2

Move disk 1 from rod 3 → rod 2

Move disk 3 from rod 1 → rod 3

Move disk 1 from rod 2 → rod 1

Move disk 2 from rod 2 → rod 3

Move disk 1 from rod 1 → rod 3

Input:

$n = 0$

Output:

0

Explanation:

When there are no disks, no moves are required.

Constraints

$0 \leq n \leq 20$

Solution:

```
class Solution {  
    public:  
        int dx[8]={-1,-1,1,1,-2,-2,2,2};  
        int dy[8]={2,-2,2,-2,-1,1,-1,1};  
        bool fillMatrix(int i,int j,int move,int n,vector<vector<int>>&result){  
            //edge case  
            if(i<0||j<0||i>=n||j>=n||result[i][j]!=-1){  
                return false;  
            }  
  
            //base case
```

```

    if(move==n*n-1){
        result[i][j]=move;
        return true;
    }
    //main logic
    result[i][j]=move;

    for(int t=0;t<8;t++){
        int newi=i+dx[t];
        int newj=j+dy[t];
        if(fillMatrix(newi,newj,move+1,n,result)==true) return true;
    }
    result[i][j]=-1;
    return false;

}

vector<vector<int>> knightTour(int n) {
    vector<vector<int>>result(n,vector<int>(n,-1));
    int i=0;
    int j=0;
    int move=0;
    if(fillMatrix(i,j,move,n,result)==true){
        return result;
    }
}

```

```
    else{  
        return {};  
    }  
}  
};
```